
Table of Contents

Acknowledgment	i
Abstract	ii
Table of Contents	iii
List of Figures	v
1 Introduction	1
1.1 Introduction	1
1.2 Summary	1
2 Literature Review	2
2.1 Literature Survey	2
2.2 Summary	3
3 Problem Formulation	4
3.1 Problem Statement	4
3.2 Objectives	4
3.3 Summary	5
4 Requirements and Methodology	6
4.1 Hardware Requirements	6
4.2 Software Requirements	6
4.3 System Overview	6
4.4 Message Overview	6
4.5 Virtual pin mapping	7
4.6 ESP32 firmware flow and logic	8
4.7 Hardware connections	8
4.8 Software and web integration	8
4.9 Summary	8
5 System Architecture	9

5.1	Overview	9
5.2	Block Diagram	9
6	Implementation	11
6.1	Introduction	11
6.2	Code for hardware implementation	11
6.3	Summary	21
7	Result and Analysis	22
7.1	Results	22
7.2	Summary	23
8	Conclusion and Scope for Future Work	26
8.1	Conclusion	26
8.2	Scope for Future Work	26
	Bibliography	27
9	Personal Profile	28

List of Figures

4.1	ESP32 pin diagram	7
5.1	System Block Diagram of IoT-Based Smart Circular Distribution System	10
7.1	Website interface for Class B	23
7.2	OLED output for Class B	24
7.3	Website interface for Class A	24
7.4	OLED output for Class A	25

1. Introduction

1.1 Introduction

Many schools, colleges, and offices still depend on manual systems of issuing items or administering any series of tasks. The results are usually delays, misunderstandings, and unequal distribution since it all depends on human handling. Due to this, simple tasks such as issuing materials, performing responsibility rotations, or even notifying users in sequence are time-consuming and not very efficient. The need for such a simple and reliable method that can manage distribution cycles automatically arises in today's world, where automation and IoT in everyday systems are increasingly used. In a circular distribution system, the entities follow a specific pattern where the tasks or items are distributed to each successive point around the circle. Automation of this process gives a guarantee of accuracy, and greatly reduces human workload, thereby assisting in smooth organization. IoT-based smart circular distribution is a project in which every single step of the circular distribution will get automated with an ESP32 microcontroller, along with a small OLED display and an IoT platform. The system can notify users, update the sequence automatically, and provide real-time status through the IoT dashboard. This makes the distribution process more systematic and easier to monitor. The proposed system is simple, cost-effective, and suited for small environments such as academic institutions and office setups. It maintains sequence orders, avoids human errors, and is easy to track by users or supervisors. The general aim of the project is to demonstrate a practical and user-friendly way of using IoT for the automation of basic operational workflows.

1.2 Summary

In traditional circular delivering system, we face some issues, every time we need someone to deliver the circular to all the classes which is time consuming. It's also a waste of paper for each circular we get, also we can't depend on personal devices because they cannot be used inside classes. Sometimes the message will be delayed in delivering to all the classes. So, to overcome this there is a need for an affordable IoT-based system ensuring timely and reliable notice delivery.

2. Literature Review

2.1 Literature Survey

Title: IoT-Based Notification and Display System for Real-Time Communication [1]

Authors: P. Sharma, R. Verma

Year: 2020

The study presents an IoT-based messaging system where a central web interface sends real-time notifications to microcontroller nodes through a cloud platform. The ESP32/NodeMCU receives the message and displays it on an LCD/OLED screen. The work demonstrates how cloud services can act as an intermediary between a web client and an embedded device, enabling message transfer without direct device-to-device communication. The research highlights advantages such as reduced manual effort, instant updates, and low hardware requirements—features closely aligned with the objective of sending structured messages from a website to an ESP32 for display purposes [1].

Title: Design of an IoT-Enabled Message Broadcasting System [2]

Authors: A. Prakash, M. R. Kumar

Year: 2021

The article discusses a centralized IoT broadcasting system where a server submits important announcements, which are then forwarded to individual IoT nodes through a cloud MQTT framework. Each node receives the message and displays it locally for users. The study focuses on simplifying communication in schools, offices, and small organizations where supervisors need a quick way to publish notifications. The architecture resembles your project concept, where a website sends a message to Blynk, Blynk forwards it to ESP32, and the ESP32 outputs the message on an OLED display. The research validates that cloud-mediated communication improves reliability and reduces dependency on manual distribution [2].

Title: ESP32 and Blynk-Based Smart Notice Board System [3]

Authors: R. Patel, S. Nair

Year: 2022

This paper presents the design and implementation of a smart notice board system using the ESP32 microcontroller and the Blynk IoT platform. The proposed system enables authorized users to send notices remotely through a mobile or web interface, which are then transmitted via the Blynk cloud to the ESP32 device. Upon receiving the data, the ESP32 displays the message on an electronic

display unit. The study emphasizes simplicity, low cost, and real-time communication, making it suitable for educational institutions and office environments. The work closely relates to the proposed project, as both systems rely on ESP32–Blynk cloud communication to deliver messages instantly to a local display unit, reducing manual effort and improving reliability [3].

Title: Cloud-Assisted IoT Communication Framework for Institutional Alert Systems [4]

Authors: S. Kumar, A. Joshi

Year: 2023

This research focuses on a cloud-assisted IoT framework designed for institutional alert and notification systems. The authors propose a centralized cloud model where alerts are generated through a web interface and delivered to IoT-enabled devices using secure cloud communication. The system supports acknowledgment feedback, ensuring that alerts are not only delivered but also confirmed by the receiving nodes. Performance analysis shows low latency and reliable message delivery across multiple devices. This work supports the design approach used in the Smart Circular Distribution System, particularly the use of cloud-based intermediaries (such as Blynk) for message transmission and acknowledgment tracking between a web application and ESP32-based hardware units [4].

2.2 Summary

The reviewed literature highlights how ESP32-based IoT systems can reliably communicate with cloud platforms such as Blynk to support real-time monitoring and remote interaction. Kakani et al. (2024) demonstrated that combining ESP32 with Blynk enables stable wireless control and instant device feedback, proving the suitability of this architecture for low-cost automation systems. Pramana et al. (2025) showed that similar IoT setups offer fast response times, dependable Wi-Fi connectivity, and smooth two-way communication between the cloud interface and the physical device. Both studies confirm the effectiveness of using ESP32 with IoT dashboards for remote commands and real-time updates—directly supporting the approach used in the present project, where messages from a website are forwarded through Blynk to the ESP32 for display on an OLED screen. These works collectively validate the feasibility, responsiveness, and practicality of the communication model implemented in this project.

3. Problem Formulation

3.1 Problem Statement

Most of the schools and colleges and even office premises follow a manual process for distributing things or managing activities in a circular order. This process usually seems to generate various problems like delays, missed turns, confusion in the sequence, and inability to track which stage the process is at the moment. In the absence of any clear system to display the order or automate the sequence, maintaining appropriate circular distribution is challenging. Moreover, supervisors cannot easily monitor or update the distribution status remotely. Manual handling increases dependency on individuals and reduces consistency. A simple automated system able to maintain the sequence, show the real-time status, and provide the possibility of remote access would help in making the process of distribution more orderly and reliable.

3.2 Objectives

- To design and implement an IoT-based automated notice and circular distribution system for classrooms.
- To enable remote monitoring and control of the circular distribution process using an IoT platform.
- To allow supervisors to view real-time updates and acknowledgments for effective management.
- To reduce manual errors involved in traditional notice circulation methods.
- To ensure accurate and sequential distribution of notices across multiple locations.
- To improve efficiency and response time within a small institutional or office environment.
- To develop a simple and low-cost system that is easy to implement.
- To design a scalable and adaptable model suitable for other sequential distribution or task-based applications.

3.3 Summary

The project deals with the challenge in message distribution at schools, colleges, and offices, which may be very difficult to handle manually. Without automation, there are possibilities of messages being delayed or missed, and there is no easy way to monitor or send updates remotely. This work has been done with the motive of developing a simple IoT-based solution using a website, Blynk, and ESP32, where messages sent from the website are delivered to the ESP32 and displayed on an OLED screen. The system should improve communication by reducing human effort and thus provide a clear and reliable way of sending and receiving important messages

4. Requirements and Methodology

4.1 Hardware Requirements

ESP32 Development Board – Microcontroller used to receive messages, control display, buzzer, and send acknowledgments, OLED Display (128×64, I2C) – Displays the circular message and sender information, Active Buzzer – Produces alert sounds for normal and urgent messages, Push Button – Allows the classroom to acknowledge the message, Jumper Wires – Connects all hardware components to the ESP32, Breadboard – Used for assembling and testing the circuit, USB Cable – Powers the ESP32 and connected modules.

4.2 Software Requirements

Arduino IDE – Used to write, compile, and upload code to ESP32, ESP32 Board Package – Enables Arduino IDE to support ESP32 programming, Blynk IoT Platform – Cloud service for sending and receiving data between web app and devices, Required Arduino Libraries – Libraries for Blynk, OLED display, and I²C communication, Web Application – Interface used by HOD to send circulars and track acknowledgments.

4.3 System Overview

Website (web UI) — Professors type a message and clicks send, Blynk Cloud — Receives the message from the website (via the Blynk API) and stores and publishes it on a virtual pin or template, ESP32 device — Connected to Blynk Cloud; subscribes to the relevant virtual pin(s). When a new message arrives, ESP32 reads it, displays it on the OLED, also sends an acknowledgement back to Blynk (which the website can read), OLED (128×64) — Local display for the message.

4.4 Message Overview

Professor's action on website, Website POSTs message to Blynk Cloud using Blynk HTTP API (web → Blynk). The payload includes: message text, target device ID. Blynk Cloud, Stores message and update a virtual pin (V0) assigned to that device. ESP32 receives message, ESP32 is connected to Wi-Fi and to Blynk. It has a handler for the virtual pin (e.g., V0). When V0 updates,

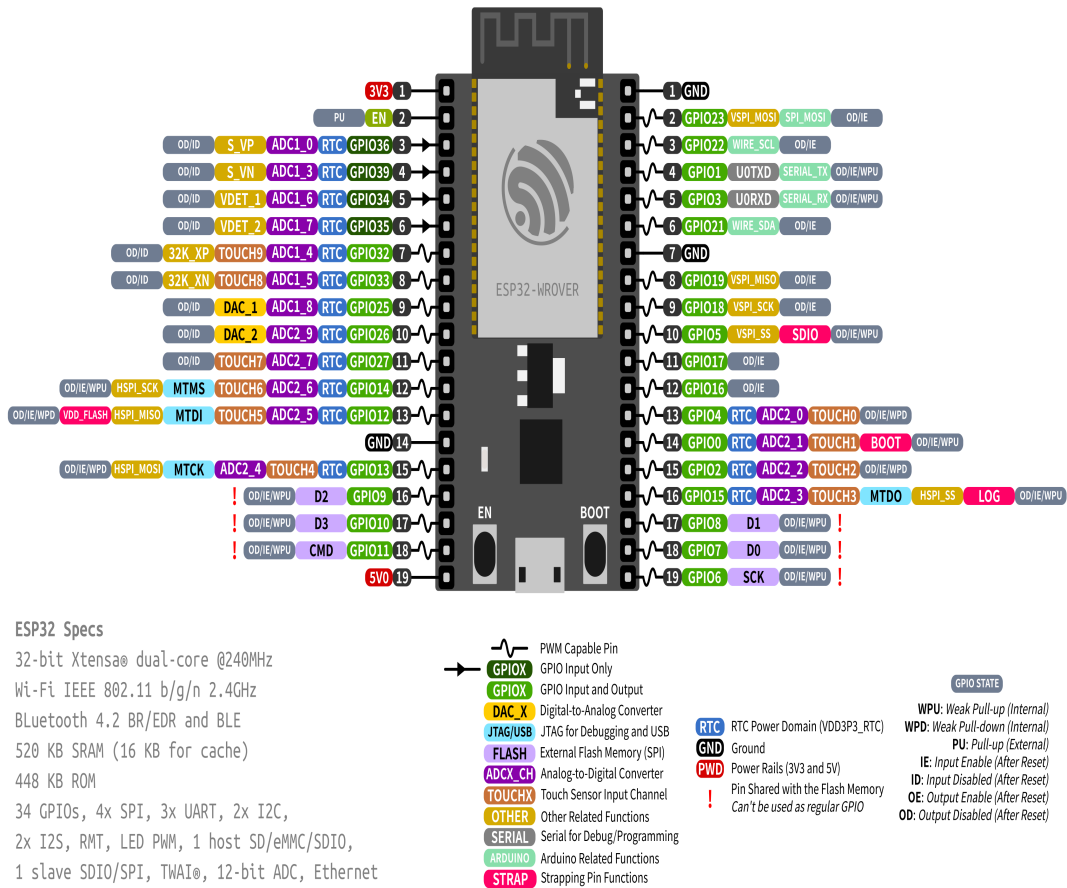


Figure 4.1: ESP32 pin diagram

the handler reads the message. ESP32 parses content, validates length of the message, and displays it on the OLED. Acknowledgement back to Blynk, ESP32 writes an acknowledgement value to another virtual pin (V3) by calling an API to confirm receipt (e.g., RECEIVED), Website reads acknowledgement, The website polls or subscribes to V1 to display delivery status to the sender.

4.5 Virtual pin mapping

V0 — Incoming message text (string)

V1 — Sender name (string)

V2 — Urgency of the message sent (integer)

V3 — Acknowledgement (integer)

4.6 ESP32 firmware flow and logic

Startup, Connect to Wi-Fi, Connect to Blynk Cloud with Auth Token, Template ID, Template name, Initialize OLED (I²C), On V0 update (message received), Validate payload (max length), If message too long: scroll the message, Display message on OLED (with simple formatting: sender, message), Reader can send acknowledgement through V3 with message ID and timestamp, Buzzer beeps briefly to notify about the message received.

4.7 Hardware connections

ESP32 — connects to Wi-Fi and Blynk; I²C pins to OLED (SDA, SCL), OLED (128×64) — I²C interface (SDA, SCL), shows message and status, Additional: buzzer and push button — buzzer for local alert when a new message arrives; push button to trigger acknowledgement.

4.8 Software and web integration

Website - Uses Blynk HTTP API to send messages to a device template. The website will be a simple HTML with some basic CSS that performs an HTTP POST. Include authentication (API token) to prevent unauthorized use, Blynk - Template created for ESP32 devices (Template ID + Auth Token). The website uses these credentials to address the proper device, ESP32 - Blynk client code (BlynkSimpleEsp32 library) with virtual pin handlers. I²C OLED library (Adafruit_SSD1306) for display text.

4.9 Summary

This chapter explains the complete working of the IoT-based Smart Circular Distribution System using ESP32 and Blynk. A web interface allows professors to send circular messages, which are routed through the Blynk Cloud to the ESP32 device. The ESP32 displays the message on an OLED, alerts users using a buzzer, and enables acknowledgment via a push button. Communication is managed using Blynk virtual pins for message data, urgency, sender details, and acknowledgment status.

5. System Architecture

5.1 Overview

The IoT-Based Smart Circular Distribution System enables digital delivery and acknowledgement of circulars using an ESP32 device. A web application allows the HOD to send messages, which are transmitted to the ESP32 through the Blynk Cloud. Upon receiving a message, the ESP32 displays the circular on an OLED screen and activates a buzzer to alert users. A push button is provided for acknowledgment, which is sent back to the web application via the cloud, ensuring reliable message delivery and confirmation.

5.2 Block Diagram

The system block diagram is shown in Figure 5.1. It consists of the following components:

- **Web Application (HOD Interface):** Used by the HOD to enter the circular message, sender name, and urgency level, which are sent to the cloud.
- **Blynk Cloud:** Acts as the communication bridge between the web application and the ESP32, forwarding messages and receiving acknowledgments.
- **ESP32 Device:** Core controller that receives data from the cloud, processes the message, controls the OLED display and buzzer, and handles acknowledgments.
- **OLED Display:** Displays the received circular along with the sender information; long messages scroll automatically.
- **Buzzer:** Provides audio alerts when a new message arrives; urgent messages trigger a stronger alert.
- **Push Button:** Allows users to acknowledge that the message has been read by sending a confirmation through the ESP32.
- **Acknowledgment to Web App:** The ESP32 sends an acknowledgment to the cloud, which updates the web application with delivery status.

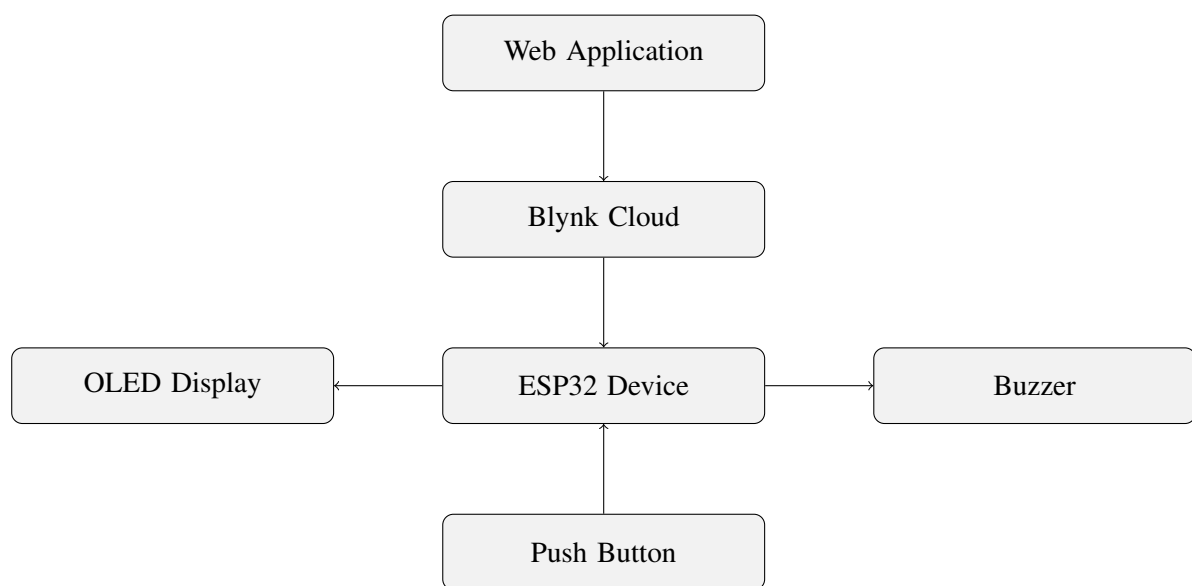


Figure 5.1: System Block Diagram of IoT-Based Smart Circular Distribution System

6. Implementation

6.1 Introduction

This chapter includes the code used for our Smart Circular Distribution System. The program is developed using the Arduino IDE and implemented on the ESP32 microcontroller. Communication between the web application and the device is achieved using the Blynk IoT platform over Wi-Fi. The code is responsible for receiving circular messages from the cloud, displaying them on the OLED display, generating alert notifications through a buzzer, and handling acknowledgments using a push button.

6.2 Code for hardware implementation

```
// ----- BLYNK TEMPLATE DETAILS -----
#define BLYNK_TEMPLATE_ID    " "
#define BLYNK_TEMPLATE_NAME  " "
#define BLYNK_AUTH_TOKEN    " "
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <vector>
using std::vector;
// Pins
#define BUZZER_PIN 25
#define BUTTON_PIN 26
// OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
// WiFi
char ssid[] = " name";
```

```

char pass[] = "password";
// App variables
String messageText = "";
String senderName = "";
int urgencyFlag = 0; // 0 normal, 1 urgent
// Message / display state
bool pendingMessage = false;
unsigned long messageArrivedTime = 0;
// V2 sync / retry control
unsigned long lastV2ReceivedTime = 0; // updated in V2 handler
unsigned long lastV2SyncCall = 0;
const unsigned long V2_SYNC_INTERVAL_MS = 200;
const unsigned long MAX_V2_WAIT_MS = 5000;
int v2SyncAttempts = 0;
const int MAX_V2_SYNC_ATTEMPTS = 25;
// Beep state machine, for buzzer
enum BeepState {BEEP_IDLE, BEEP_ON_1, BEEP_GAP, BEEP_ON_2, BEEP_DONE};
BeepState beepState = BEEP_IDLE;
unsigned long beepStateTs = 0;
unsigned long beepDoneTs = 0;
// Scrolling / paragraph layout
vector<String> wrappedLines;
int lineHeight = 8; // px per line for textSize=1 (approx)
int headerHeight = 20; // increased so message starts lower (avoids header over
int headerTextY = 4; // header text vertical offset inside header area
int areaY = headerHeight; // message area start Y
int areaHeight = SCREEN_HEIGHT - headerHeight; // available height
int blockHeight = 0; // total pixel height of wrapped block
int scrollY = 0; // current vertical offset (px)
bool verticalScrolling = false;
unsigned long lastScrollMs = 0;
const unsigned long SCROLL_INTERVAL_MS = 30; // 25-50ms good; smaller ->
smoother/faster
const int SCROLL_STEP_PX = 1;
unsigned long pauseAtEndMs = 800; // pause when full scroll reached
unsigned long endPauseStart = 0;

```

```

bool ackSent = false;
// ----- BLYNK handlers -----
BLYNK_WRITE(V0) {
  messageText = param.asStr();
  Serial.print("V0 (Message) received: ");
  Serial.println(messageText);
  // Request latest urgency
  Serial.println("V0 handler: calling Blynk.syncVirtual(V2)");
  Blynk.syncVirtual(V2);
  pendingMessage = true;
  messageArrivedTime = millis();
  v2SyncAttempts = 0;
  lastV2SyncCall = millis();
  ackSent = false;
  Blynk.virtualWrite(V3, 0);
  BLYNK_WRITE(V1) {
    senderName = param.asStr();
    Serial.print("V1 (Sender) received: ");
    Serial.println(senderName);
    // do not prepare display here
  }
  BLYNK_WRITE(V2) {
    urgencyFlag = param.asInt();
    lastV2ReceivedTime = millis();
    Serial.print("V2 (Urgency) received: ");
    Serial.print(urgencyFlag);
    Serial.print(" ts=");
    14
    Serial.println(lastV2ReceivedTime);
  }
  void setup() {
    Serial.begin(115200);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
      Serial.println("OLED not found!");
    }
  }
}

```



```

while (1);
}
display.setTextWrap(false);
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0, 20);
display.println("Connecting...");
display.display();
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
display.clearDisplay();
display.setCursor(0, 20);
display.println("Connected!");
display.display();
Serial.println("Setup complete.");
}
void loop() {
  Blynk.run();
  // handle button ack
  if (digitalRead(BUTTON_PIN) == LOW && !ackSent) {
    ackSent = true;
    Blynk.virtualWrite(V3, 1);
    Serial.println("Button pressed -> ack sent (V3=1)");
    delay(250); // small debounce
  }
  // If a new message arrived and beep hasn't started yet, try to ensure we have
  if (pendingMessage && beepState == BEEP_IDLE) {
    bool v2ArrivedAfterMsg = (lastV2ReceivedTime >= messageArrivedTime);
    if (v2ArrivedAfterMsg) {
      Serial.println("V2 is fresh -> starting beep.");
      startBeepSequence();
    } else {
      unsigned long now = millis();
      if ((now - lastV2SyncCall >= V2_SYNC_INTERVAL_MS) && (v2SyncAttempts <
MAX_V2_SYNC_ATTEMPTS)) {
        Serial.print("Attempting Blynk.syncVirtual(V2) (attempt ");

```

```

Serial.print(v2SyncAttempts + 1);
Serial.println(" ");
Blynk.syncVirtual(V2);
v2SyncAttempts++;
lastV2SyncCall = now;
}
if (now - messageArrivedTime >= MAX_V2_WAIT_MS) {
Serial.println("V2 wait exceeded -> start beep with fallback.");
startBeepSequence();
} } }
// manage beep state machine
handleBeepState();
// after beep done: wait 5s then prepare display
if (beepState == BEEP_DONE) {
if (beepDoneTs == 0) beepDoneTs = millis();
if (millis() - beepDoneTs >= 5000) {
// prepare layout & start display/scrolling
layoutMessageBlock();
pendingMessage = false;
beepDoneTs = 0;
beepState = BEEP_IDLE;
} }
// handle vertical scrolling if needed
if (verticalScrolling && wrappedLines.size() > 0) {
unsigned long now = millis();
if (endPauseStart) {
if (now - endPauseStart >= pauseAtEndMs) {
// restart loop
endPauseStart = 0;
scrollY = 0; // reset to top
redrawMessageBlock();
}
} else if (now - lastScrollMs >= SCROLL_INTERVAL_MS) {
lastScrollMs = now;
scrollY -= SCROLL_STEP_PX;
// if we've scrolled full amount (negative)

```

```

if (scrollY <= (areaHeight - blockHeight)) {
// reached the end | pause then restart
endPauseStart = now;
} else {
redrawMessageBlock();
} } } }
// ----- beep helpers -----
void startBeepSequence() {
Serial.print("startBeepSequence(); urgency=");
Serial.println(urgencyFlag);
beepState = BEEP_ON_1;
beepStateTs = millis();
digitalWrite(BUZZER_PIN, HIGH);
}
void handleBeepState() {
unsigned long now = millis();
switch (beepState) {
case BEEP_IDLE:
case BEEP_DONE:
break;
case BEEP_ON_1:
if (now - beepStateTs >= 1500) {
digitalWrite(BUZZER_PIN, LOW);
if (urgencyFlag == 0) {
beepState = BEEP_DONE;
beepStateTs = now;
beepDoneTs = now;
Serial.println("normal beep done");
} else {
beepState = BEEP_GAP;
beepStateTs = now;
Serial.println("gap after 1st");
} }
break;
case BEEP_GAP:
if (now - beepStateTs >= 400) {

```

```

digitalWrite(BUZZER_PIN, HIGH);
beepState = BEEP_ON_2;
beepStateTs = now;
Serial.println("start 2nd beep");
}
break;
case BEEP_ON_2:
if (now - beepStateTs >= 1500) {
digitalWrite(BUZZER_PIN, LOW);
beepState = BEEP_DONE;
beepStateTs = now;
beepDoneTs = now;
Serial.println("urgent beep done");
}
break;
} }
// ----- text layout & drawing -----
// word-wrap into lines that fit SCREEN_WIDTH for current text size
void wrapTextToLines(const String &text, vector<String> &outLines) {
outLines.clear();
String trimmed = text;
trimmed.trim();
// replace CR/LF with spaces
trimmed.replace("\r", " ");
trimmed.replace("\n", " ");
// split words by spaces
int len = trimmed.length();
int i = 0;
String word = "";
vector<String> words;
while (i < len) {
char c = trimmed[i];
if (c == ' ') {
if (word.length() > 0) {
words.push_back(word);
word = "";

```

```

}
// preserve single space as separator
} else {
word += c;
}
i++;
}
if (word.length() > 0) words.push_back(word);
String line = "";
for (size_t widx = 0; widx < words.size(); ++widx) {
String &wrd = words[widx];
// candidate if we add this word to current line
String candidate = (line.length() == 0) ? wrd : (line + " " + wrd);
// measure candidate width
int16_t x1, y1;
uint16_t candW, candH;
display.getTextBounds(candidate.c_str(), 0, 0, &x1, &y1, &candW, &candH);
// if too wide and line empty -> break the word itself into pieces
if (candW > SCREEN_WIDTH && line.length() == 0) {
// break the word into character chunks that fit
String part = "";
for (int ci = 0; ci < wrd.length(); ++ci) {
part += wrd[ci];
display.getTextBounds(part.c_str(), 0, 0, &x1, &y1, &candW, &candH);
if (candW > SCREEN_WIDTH) {
// remove last char
part.remove(part.length() - 1);
outLines.push_back(part);
part = String(wrd[ci]); // start new part with current char
} }
if (part.length() > 0) outLines.push_back(part);
line = ""; // start new line after word parts
} else if (candW <= SCREEN_WIDTH) {
// ok to add
line = candidate;
} else {

```

```

// candidate too wide, push current line and start new line with word
if (line.length() > 0) outLines.push_back(line);
line = wrd;
}
}
if (line.length() > 0) outLines.push_back(line);
}
// prepare wrappedLines, determine verticalScrolling, blockHeight, reset scro
void layoutMessageBlock() {
// ensure font settings
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setTextWrap(false);
wrapTextToLines(messageText, wrappedLines);
// compute lineHeight using getTextBounds of "Tg" as approximate height
int16_t x1, y1;
uint16_t w, h;
display.getTextBounds("Tg", 0, 0, &x1, &y1, &w, &h);
lineHeight = max(8, (int)h); // ensure at least 8
areaY = headerHeight;
areaHeight = SCREEN_HEIGHT - headerHeight;
blockHeight = (int)wrappedLines.size() * lineHeight;
Serial.println("----- layoutMessageBlock -----");
Serial.print("lines = "); Serial.println((int)wrappedLines.size());
Serial.print("lineHeight = "); Serial.println(lineHeight);
Serial.print("blockHeight = "); Serial.println(blockHeight);
Serial.print("areaHeight = "); Serial.println(areaHeight);
if (blockHeight <= areaHeight) {
verticalScrolling = false;
scrollY = 0;
// draw static immediately
drawStaticBlock();
Serial.println("verticalScrolling = false (fits)");
} else {
verticalScrolling = true;
scrollY = 0; // start at top (0). We'll scroll negative.

```

```

lastScrollMs = millis();
endPauseStart = 0;
// draw initial frame
drawStaticBlock(); // draws header and first visible part
Serial.println("verticalScrolling = true (will scroll)");
} }
// draw header + static block at current scrollY (if scrollY==0 it draws top)
void drawStaticBlock() {
display.clearDisplay();
// header (draw slightly lower inside the header band)
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0, headerTextY);
display.print("From: ");
display.println(senderName);
// message area
int y = areaY + scrollY; // initial y offset
display.setTextSize(1);
for (size_t i = 0; i < wrappedLines.size(); ++i) {
if (y + lineHeight < areaY) {
// line is above visible area; skip
y += lineHeight;
continue;
}
if (y >= SCREEN_HEIGHT) {
// below visible area; then break
break;
}
display.setCursor(0, y);
display.print(wrappedLines[i]);
y += lineHeight;
}
display.display();
}
// redraw during scrolling (uses current scrollY)
void redrawMessageBlock() {

```

```

// keep header intact, clear only message area
display.fillRect(0, areaY, SCREEN_WIDTH, areaHeight, SSD1306_BLACK);
int y = areaY + scrollY;
for (size_t i = 0; i < wrappedLines.size(); ++i) {
    if (y + lineHeight < areaY) {
        y += lineHeight; continue;
    }
    if (y >= SCREEN_HEIGHT) break;
    display.setCursor(0, y);
    display.print(wrappedLines[i]);
    y += lineHeight;
}
display.display();
}

```

6.3 Summary

The code was successfully compiled, uploaded, and tested on the ESP32 hardware. Proper reception of messages from the cloud, correct display on the OLED, and alert generation based on message urgency were observed. The acknowledgment mechanism using the push button functioned correctly, confirming that the system operates reliably as intended.

7. Result and Analysis

7.1 Results

The experimental results show that the IoT-Based Smart Circular Distribution System performs reliably across multiple classroom units. All ESP32 devices consistently received circular messages, generated appropriate alerts, displayed content correctly on the OLED, and returned acknowledgments with low latency. The system maintained stable performance under varying message urgency levels, demonstrating its suitability for real-time academic communication.

Message Delivery Performance

The ESP32 devices successfully received circular messages from the web interface through Blynk Cloud with no packet loss observed during testing. The sender's name and message text were displayed correctly on the OLED display. Even when multiple ESP32 devices were active, all units received the same message simultaneously without delay, confirming that the virtual pin mapping handled multi-device communication effectively.

OLED Display Functionality

Short messages were displayed instantly and clearly. Long messages triggered the scrolling feature, which ensured full readability without truncation. Display updates occurred after the alert delay, allowing students adequate time to approach and read the message.

Buzzer Alert Performance

Normal messages produced a single beep. Urgent messages triggered the designated double-beep pattern, clearly distinguishing priority messages.

Acknowledgment Functionality

Pressing the push button reliably updated Virtual Pin V3 in Blynk Cloud. The web interface correctly reflected acknowledgment with a double-tick indicator, confirming that the circular was received and read. Acknowledgments were transmitted within 1–2 seconds, validating the low-latency nature of ESP–cloud communication.

System Reliability and Scalability

The system operated continuously without crashes or communication failures. Multiple devices were able to function independently under the same cloud environment using shared virtual-pin definitions. Configuration allows effortless addition of new ESP32 classroom units, making the system scalable for entire academic institutions.

7.2 Summary

The testing results indicate that the IoT-Based Smart Circular Distribution System operates reliably in a classroom environment. Messages were delivered correctly to multiple ESP32 devices with proper display, alert indication, and scrolling functionality. The acknowledgment mechanism functioned with low latency, confirming effective communication between the devices and the web interface.

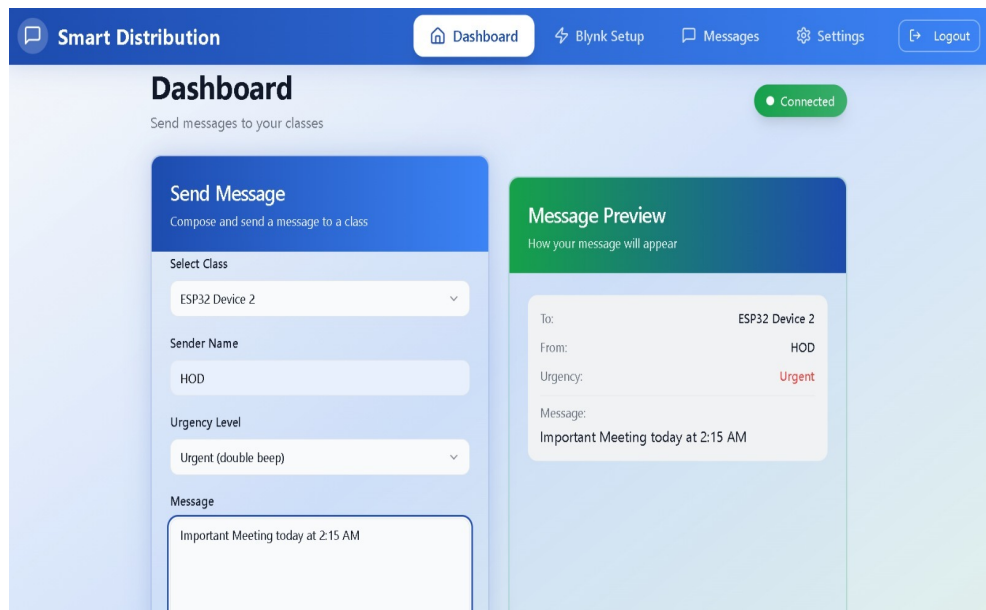


Figure 7.1: Website interface for Class B

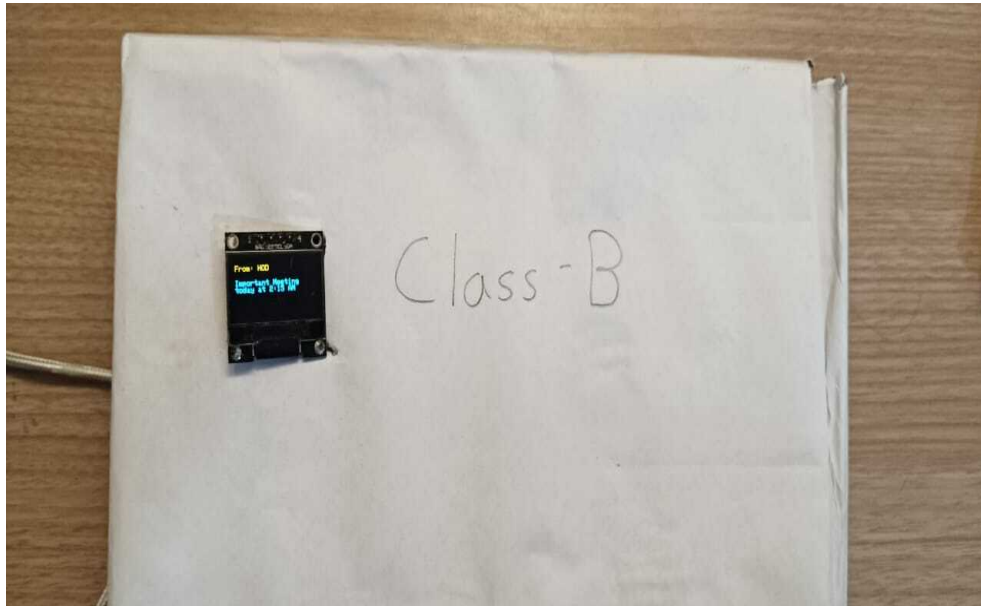


Figure 7.2: OLED output for Class B

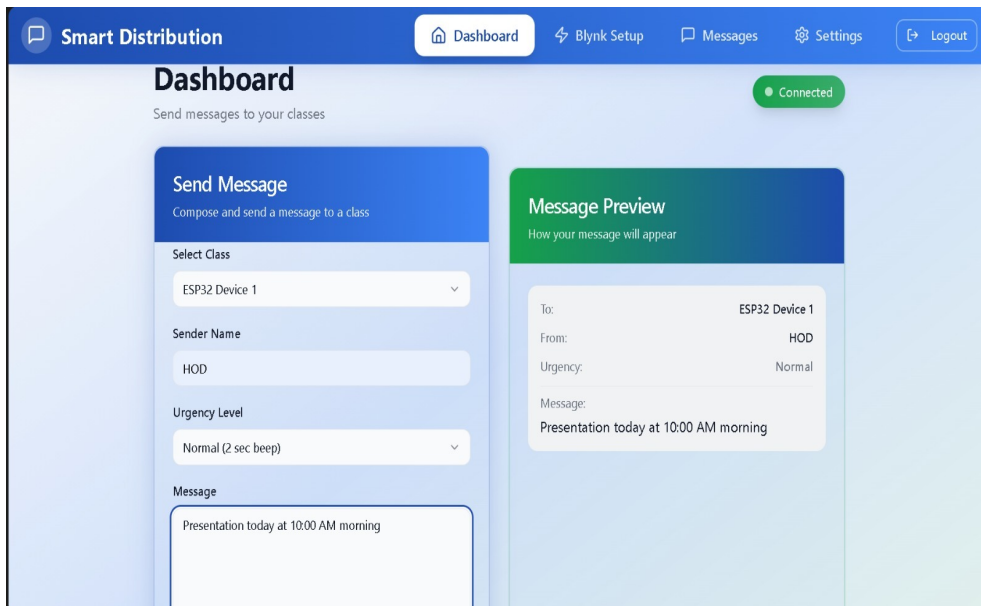


Figure 7.3: Website interface for Class A

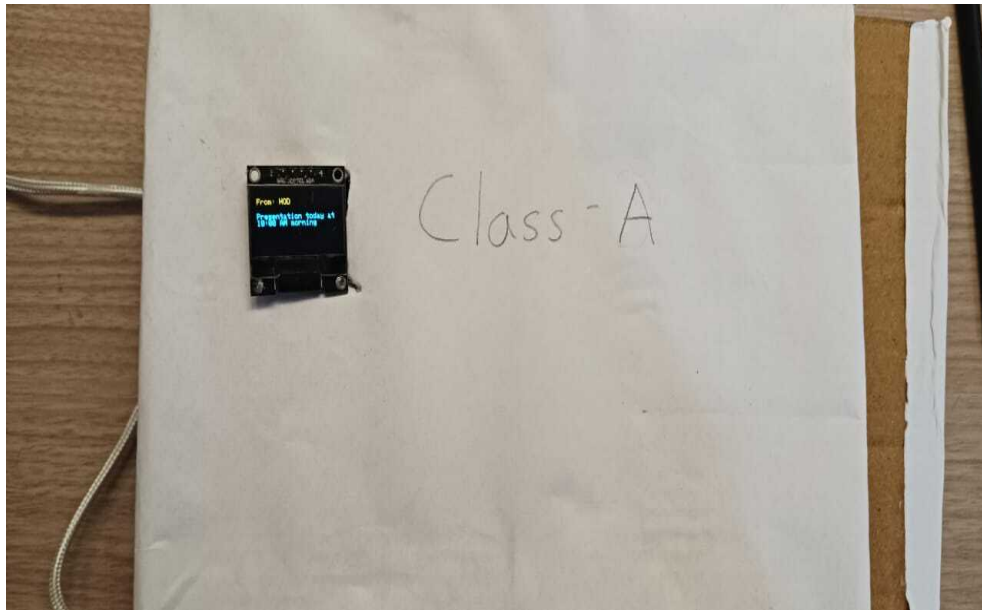


Figure 7.4: OLED output for Class A

8. Conclusion and Scope for Future Work

8.1 Conclusion

The IoT-Based Smart Circular Distribution System successfully automates the process of disseminating circulars across campus classrooms. By integrating ESP32 microcontrollers, Blynk Cloud services, and a custom web interface, the system provides an efficient, real-time communication channel between the HOD and classroom units. The project has achieved its primary goals: Instant message delivery, Classification of circulars based on urgency, Audible alert generation, Clear on-screen message display with auto-scrolling, Reliable acknowledgment feedback, Overall, the developed system reduces dependency on manual circular distribution, minimizes delays, and enhances communication efficiency within educational institutions. The design is modular, cost-effective, and scalable, making it a strong candidate for real-world deployment.

8.2 Scope for Future Work

Voice Output Instead of Only Display- The system can be upgraded to convert the received text message into speech using a small text to-speech module, allowing the ESP32 to announce the message aloud instead of showing it only on the OLED.

Mobile App Shortcut for Quick Sending- A simple dedicated mobile app or shortcut button could be added so users can send messages faster without opening the full website every time.

Multiple ESP32 Units Receiving the Same Message- The system can be extended so that the website sends one message, and multiple ESP32 units in different rooms receive and display/announce it simultaneously.

Bibliography

- [1] P. Sharma and R. Verma, "Iot-based notification and display system for real-time communication," *International Journal of Engineering Research and Technology*, vol. 9, no. 6, pp. 112–118, 2020.
- [2] A. Prakash and M. R. Kumar, "Design of an iot-enabled message broadcasting system," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 10, no. 3, pp. 45–51, 2021.
- [3] R. Patel and S. Nair, "Esp32 and blynk-based smart notice board system," *International Journal of Innovative Technology and Exploring Engineering*, vol. 11, no. 8, pp. 23–29, 2022.
- [4] S. Kumar and A. Joshi, "Cloud-assisted iot communication framework for institutional alert systems," *IEEE Access*, vol. 11, pp. 55 840–55 850, 2023.